

1) Configuración del Entorno de Desarrollo

Para poder ejecutar esta API en tu computador, necesitas instalar y configurar lo siguiente:

✓ Requisitos previos

Herramienta	Versión sugerida	¿Para qué sirve?
Node.js	v18 o superior	Ejecuta JavaScript en el servidor
npm	v9 o superior	Administrador de paquetes (viene con Node)
MySQL	v8 o superior	Base de datos relacional
Git (opcional)	última estable	Control de versiones
Postman o Insomnia (opcional)	última estable	Probar los endpoints de la API

📦 Instalación paso a paso

1. Instalar Node.js y npm

- Ve a <https://nodejs.org>
- Descarga la versión LTS (recomendada)
- Ejecuta el instalador y sigue los pasos
- Verifica la instalación:

```
bash
node --version
npm --version
```

2. Instalar MySQL

- **Windows/Mac:** Descarga desde <https://dev.mysql.com/downloads/installer/>
- **Linux (Ubuntu/Debian):**

```
bash
sudo apt update
sudo apt install mysql-server
```

- Inicia el servicio MySQL
- Crea un usuario y contraseña (ej: `root` / `tu_password`)

3. Instalar herramientas adicionales (opcional)

```
bash
# Nodemon global (reinicia automáticamente el servidor)
npm install -g nodemon

# Cliente MySQL para pruebas
npm install -g mysql2
```

🗄️ Configurar la Base de Datos

1. Conéctate a MySQL:

```
bash
mysql -u root -p
```

2. Crea la base de datos:

```
sql

CREATE DATABASE IF NOT EXISTS api_users;
USE api_users;
```

3. Crea la tabla `users`:

```
sql

CREATE TABLE `users` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `lastname` VARCHAR(100) NOT NULL,
  `email` VARCHAR(150) NOT NULL UNIQUE,
  `password` VARCHAR(255) NOT NULL,
  `phone` VARCHAR(20) DEFAULT NULL,
  `image` VARCHAR(255) DEFAULT NULL,
  `role` ENUM('admin', 'seller', 'customer', 'user') DEFAULT 'user',
  `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
);
```

4. (Opcional) Inserta un usuario admin de prueba:

```
sqli
```

```
INSERT INTO users (name, lastname, email, password, phone, role)
VALUES ('Admin', 'Principal', 'admin@test.com', '$2a$10$N9qo8uL0ickgx2ZMRZoMy.Mq
rqk7qZxXqKqKqKqKqKqKqKqKqKq', '123456789', 'admin');
```

⚠ La contraseña encriptada es para 123456

📁 Estructura esperada del proyecto

Cuando clones o crees el proyecto, debes tener esta estructura:

```

text

backend/
├─ nodejs_express/
│   │ config/
│   │ │ config.js
│   │ │ keys.js
│   │ │ passport.js
│   │ └─ swagger.js
│   └─ controllers/
│       │ userController.js
│   └─ middlewares/
│       │ authMiddleware.js
│   └─ models/
│       │ user.js
│   └─ routes/
│       │ userRoutes.js
│   └─ .env
│   └─ index.js
│   └─ package.json
│   └─ package-lock.json
│   └─ server.js

```

Archivo `.env` obligatorio

Crea un archivo `.env` en la raíz del proyecto (`backend/nodejs_express/.env`) con:

```
env

# Puerto y host
PORT=3000
HOST=localhost

# Base de datos MySQL
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña
DB_NAME=api_users
DB_PORT=3306

# JWT (JSON Web Token)
JWT_SECRET=tu_clave_secreta_muy_segura_cambia_esto
```

⚠ **Importante:** No subas el archivo `.env` a GitHub (debe estar en `.gitignore`)

✅ Verificar que todo está listo

Ejecuta estos comandos para asegurarte:

```
bash

node --version      # v18.x o superior
npm --version       # v9.x o superior
mysql --version     # v8.x o superior
```

Si todo está correcto, **pasamos al punto 2** cuando me indiques "siguiente".

siguiente

Perfecto. Continuamos con el segundo punto.

2) Iniciar el Proyecto (todos los pasos para correr la API)

Una vez que tienes todo el entorno configurado, sigue estos pasos para poner en marcha la API.

Paso 1: Obtener el proyecto

Si ya tienes los archivos, colócate en la carpeta del proyecto:

```
bash

cd backend/nodejs_express
```

Si estás empezando desde cero, crea la estructura de carpetas y copia los archivos que mostraste anteriormente.

Paso 2: Instalar dependencias

Ejecuta el siguiente comando dentro de la carpeta `nodejs_express`:

```
bash
```

```
npm install
```

Esto instalará todas las dependencias listadas en `package.json` :

Dependencia	¿Para qué sirve?
<code>express</code>	Framework web para Node.js
<code>mysql</code>	Conector a base de datos MySQL
<code>bcryptjs</code>	Encriptar contraseñas
<code>jsonwebtoken</code>	Generar y verificar tokens JWT
<code>cors</code>	Permitir peticiones desde otros dominios
<code>dotenv</code>	Cargar variables de entorno desde <code>.env</code>
<code>morgan</code>	Logging de peticiones HTTP
<code>passport</code> + <code>passport-jwt</code>	Autenticación con JWT
<code>swagger-jsdoc</code> + <code>swagger-ui-express</code>	Documentación de la API
<code>nodemon</code> (dev)	Reinicio automático del servidor

Paso 3: Verificar la configuración de la base de datos

Asegúrate de que:

1. MySQL esté corriendo:

```
bash

# En Windows (services.msc) o:
net start MySQL

# En Linux/Mac:
sudo systemctl status mysql
```

2. Las credenciales del `.env` sean correctas:

```
env

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña_real
DB_NAME=api_users
DB_PORT=3306
```

3. La tabla `users` exista (la creaste en el paso anterior)

Paso 4: Ejecutar la API

Tienes dos opciones:

Opción A: Modo desarrollo (con nodemon - recomendado)

```
bash

npm run dev
```

- Nodemon reinicia automáticamente el servidor cuando detecta cambios
- Ideal para desarrollo

Opción B: Modo producción

```
bash

npm start
```

- Ejecuta `node server.js`
- No se reinicia automáticamente

✅ Paso 5: Verificar que todo funciona

Si todo está correcto, deberías ver en la consola:

```
text

Base de datos conectada
Servidor corriendo en http://localhost:3000
Accesible en la red local en http://192.168.x.x:3000
```

🔧 Paso 6: Probar la API (prueba rápida)

Abre otra terminal o usa Postman para probar:

Prueba 1: Endpoint raíz

```
bash

curl http://localhost:3000/
```

Respuesta esperada:

```
text

Ruta raíz del Backend
```

Prueba 2: Endpoint de test

```
bash

curl http://localhost:3000/test
```

Respuesta esperada:

```
text

Ruta TEST
```

Prueba 3: Registrar un usuario

```
bash

curl -X POST http://localhost:3000/api/users/create \
-H "Content-Type: application/json" \
-d '{
  "name": "Juan",
  "lastname": "Perez",
  "email": "juan@test.com",
  "password": "123456",
  "phone": "123456789"
}'
```

Prueba 4: Iniciar sesión

```
bash
```

```
curl -X POST http://localhost:3000/api/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "juan@test.com",
  "password": "123456"
}'
```

Esto te devolverá un token JWT que usarás para rutas protegidas.

Prueba 5: Ruta protegida (requiere token)

```
bash

curl http://localhost:3000/api/users/ \
-H "Authorization: Bearer TU_TOKEN_AQUI"
```

Posibles errores y soluciones

Error	Causa probable	Solución
Cannot find module 'express'	Faltan dependencias	Ejecuta <code>npm install</code>
Access denied for user 'root'	Contraseña incorrecta en <code>.env</code>	Verifica <code>DB_PASSWORD</code>
Unknown database 'api_users'	No existe la BD	Créala con <code>CREATE DATABASE</code>
Port 3000 already in use	Puerto ocupado	Cambia <code>PORT</code> en <code>.env</code> o mata el proceso
JWT secret not set	Falta <code>JWT_SECRET</code> en <code>.env</code>	Agrega la variable

Resumen de comandos útiles

```
bash

# Instalar dependencias
npm install

# Ejecutar en modo desarrollo (con nodemon)
npm run dev

# Ejecutar en modo producción
npm start

# Instalar una nueva dependencia
npm install nombre_paquete

# Ver dependencias instaladas
npm list --depth=0
```

¿Vamos al **siguiente** punto para explicar la estructura de carpetas y archivos?

siguiente

Perfecto. Vamos con el tercer punto.

3) Estructura de carpetas y archivos con su respectiva explicación

Aquí tienes la estructura completa del proyecto con la función de cada carpeta y archivo:

```
text

backend/
├── nodejs_express/                # Raíz del proyecto API
│   ├── config/                   # Configuraciones globales
│   │   ├── config.js             # Conexión a MySQL
│   │   ├── keys.js              # Claves secretas (JWT)
│   │   ├── passport.js          # Estrategia de autenticación JWT
│   │   └── swagger.js           # Documentación de API (vacío)
│   ├── controllers/             # Lógica de negocio
│   │   └── userController.js     # Controlador de usuarios (CRUD + login)
│   ├── middlewares/             # Funciones intermedias
│   │   └── authMiddleware.js     # Verificación de token y roles
│   ├── models/                 # Interacción con BD
│   │   └── user.js              # Modelo de usuario (consultas SQL)
│   ├── routes/                 # Definición de endpoints
│   │   └── userRoutes.js        # Rutas de usuarios (públicas/protegidas)
│   ├── .env                    # Variables de entorno (NO subir a Git)
│   ├── index.js                # Punto de entrada (crea el servidor HTTP)
│   └── server.js               # Configuración de Express (middlewares, ruta
s)
├── package.json                # Dependencias y scripts
└── package-lock.json          # Versiones exactas de dependencias
```

Carpeta `config/` - Configuraciones

Archivo	Función	¿Qué hace exactamente?
<code>config.js</code>	Conexión a MySQL	Crea y exporta la conexión a la base de datos usando variables de entorno
<code>keys.js</code>	Clave JWT	Exporta la clave secreta para firmar/verificar tokens JWT
<code>passport.js</code>	Autenticación JWT	Configura Passport para validar tokens JWT en peticiones protegidas
<code>swagger.js</code>	Documentación	Archivo <code>预留</code> para documentar la API con Swagger (actualmente vacío)

Carpeta `controllers/` - Lógica de negocio

Archivo	Función	¿Qué hace exactamente?
<code>userController.js</code>	Controlador de usuarios	Contiene 6 funciones: <code>login</code> , <code>getAllUsers</code> , <code>getUserById</code> , <code>register</code> , <code>getUserUpdate</code> , <code>getUserDelete</code> . Cada una recibe la petición (req) y respuesta (res), procesa los datos y devuelve JSON

📁 Carpeta `middlewares/` - Funciones intermedias

Archivo	Función	¿Qué hace exactamente?
<code>authMiddleware.js</code>	Autenticación y autorización	Contiene <code>verifyToken</code> (valida JWT) y <code>authorizeRoles</code> (verifica si el usuario tiene el rol permitido). Se ejecutan ANTES de llegar al controlador

📁 Carpeta `models/` - Modelos (capa de datos)

Archivo	Función	¿Qué hace exactamente?
<code>user.js</code>	Modelo de usuario	Contiene 5 métodos: <code>findAll</code> , <code>findById</code> , <code>findByEmail</code> , <code>create</code> , <code>update</code> , <code>delete</code> . Cada uno ejecuta consultas SQL en MySQL. NUNCA interactúa directamente con la solicitud HTTP

📁 Carpeta `routes/` - Definición de endpoints

Archivo	Función	¿Qué hace exactamente?
<code>userRoutes.js</code>	Rutas de usuarios	Define los endpoints HTTP (GET, POST, PUT, DELETE) y asigna qué middleware y controlador ejecutar para cada uno

📄 Archivos raíz

Archivo	Función	¿Qué hace exactamente?
<code>index.js</code>	Punto de entrada	Crea el servidor HTTP, configura CORS, define puerto y host, y arranca la API
<code>server.js</code>	Configuración Express	Configura middlewares globales (morgan, json, cors, etc.), define rutas base y manejo de errores
<code>package.json</code>	Manifiesto del proyecto	Lista dependencias, scripts (<code>start</code> , <code>dev</code> , <code>test</code>), metadatos del proyecto
<code>.env</code>	Variables de entorno	Almacena datos sensibles (credenciales BD, JWT_SECRET, puerto) - NUNCA subir a Git

🔄 Flujo de petición (cómo se relacionan los archivos)

Cuando un cliente hace una petición, el flujo es:

```
text

1. Cliente → index.js (servidor escucha en puerto 3000)
  ↓
2. server.js (middlewares globales: morgan, cors, json)
  ↓
3. userRoutes.js (coincide la ruta /api/users/...)
  ↓
4. authMiddleware.js (verifyToken, authorizeRoles) - SÓLO rutas protegidas
  ↓
```



```
5. userController.js (función específica: getAllUsers, etc.)
  ↓
6. user.js model (consulta SQL: findAll, findById, etc.)
  ↓
7. MySQL → Respuesta → Controller → Cliente (JSON)
```

📊 Tabla de responsabilidades (arquitectura MVC)

Capa	Archivo(s)	Responsabilidad
Vista (View)	No aplica (API REST)	Devuelve JSON, no HTML
Controlador (Controller)	<code>userController.js</code>	Procesar petición, llamar al modelo, formatear respuesta
Modelo (Model)	<code>models/user.js</code>	Ejecutar consultas SQL, interactuar con BD
Rutas (Routes)	<code>routes/userRoutes.js</code>	Definir endpoints y asignar controladores
Middleware	<code>authMiddleware.js</code>	Autenticar, autorizar, validar

📁 Convenciones de nombres

Tipo	Convención	Ejemplo
Archivos de configuración	<code>nombre.config.js</code> O <code>nombre.js</code>	<code>config.js</code> , <code>passport.js</code>
Controladores	<code>nombreController.js</code>	<code>userController.js</code>
Modelos	<code>nombre.js</code> (singular)	<code>user.js</code>
Rutas	<code>nombreRoutes.js</code>	<code>userRoutes.js</code>
Middleware	<code>nombreMiddleware.js</code>	<code>authMiddleware.js</code>
Variables en .env	<code>MAYÚSCULAS_SNAKE_CASE</code>	<code>DB_HOST</code> , <code>JWT_SECRET</code>

⚠️ Archivos importantes que NO ves pero existen

Archivo	Ubicación	Función
<code>node_modules/</code>	Raíz del proyecto	Contiene TODAS las dependencias instaladas (no se sube a Git)
<code>package-lock.json</code>	Raíz del proyecto	Bloquea versiones exactas de dependencias
<code>.gitignore</code>	Raíz del proyecto	Ignora <code>node_modules/</code> , <code>.env</code> , etc.

¿Listo para el siguiente punto, donde explicaremos el código archivo por archivo?

siguiente

Perfecto. Llegamos al punto más detallado.

4) Código del Proyecto (explicación archivo por archivo)

Vamos a explicar cada archivo desde el más profundo (modelos/configuraciones) hasta el más externo (index.js), con análisis paso a paso.

Archivo: config/config.js

Ubicación: backend/nodejs_express/config/config.js

Función: Establecer la conexión con la base de datos MySQL

Código:

```
javascript

require('dotenv').config();
const mysql = require('mysql');
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT
});
db.connect(function(err) {
  if (err) throw err;
  console.log('Base de datos conectada')
});
module.exports = db;
```

Explicación línea por línea:

Línea	Explicación
<code>require('dotenv').config();</code>	Carga las variables de entorno desde el archivo <code>.env</code> a <code>process.env</code>
<code>const mysql = require('mysql');</code>	Importa el paquete mysql para conectar con MySQL
<code>mysql.createConnection({...})</code>	Crea un objeto de conexión con los datos de <code>.env</code> (host, usuario, contraseña, base de datos, puerto)
<code>db.connect(function(err) {...})</code>	Intenta conectar a MySQL. Si hay error, lanza excepción. Si éxito, muestra mensaje
<code>module.exports = db;</code>	Exporta la conexión para que otros archivos (como <code>models/user.js</code>) la usen

Archivo: config/keys.js

Ubicación: backend/nodejs_express/config/keys.js

Función: Exportar la clave secreta para JWT

Código:

```
javascript

require('dotenv').config();

module.exports = {
  secretOrKey: process.env.JWT_SECRET
};
```

Explicación:

Línea	Explicación
<code>require('dotenv').config();</code>	Carga variables de entorno
<code>module.exports = { secretOrKey: ... }</code>	Exporta un objeto con la propiedad <code>secretOrKey</code> que contiene el valor de <code>JWT_SECRET</code> del <code>.env</code>
<code>process.env.JWT_SECRET</code>	Lee la clave secreta (ej: "mi_secreto_super_seguro")

Archivo: `config/passport.js`

Ubicación: `backend/nodejs_express/config/passport.js`

Función: Configurar la estrategia de autenticación JWT con Passport

Código:

```
javascript

const JwtStrategy = require('passport-jwt').Strategy;
const ExtractJwt = require('passport-jwt').ExtractJwt;
const passport = require('passport');
const Keys = require('./keys');
const User = require('../models/user');

const opts = {
  jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  secretOrKey: Keys.secretOrKey
};

passport.use(new JwtStrategy(opts, (jwt_payload, done) => {
  User.findById(jwt_payload.id, (err, user) => {
    if (err) {
      return done(err, false);
    }
    if (user) {
      return done(null, user);
    }
    else{
      return done(null, false);
    }
  });
}));

module.exports = passport;
```

Explicación paso a paso:

Bloque	Explicación
Importaciones	<code>JwtStrategy</code> = la estrategia JWT; <code>ExtractJwt</code> = cómo extraer el token; <code>passport</code> = middleware de autenticación
<code>opts</code>	Configuración: el token viene del header <code>Authorization: Bearer <token></code> y usa la clave secreta para verificarlo
<code>passport.use()</code>	Registra la estrategia. Cuando llegue un token válido, ejecuta el callback
Callback	Recibe <code>jwt_payload</code> (datos del token: id, email, role) y <code>done</code> (función para devolver resultado)
<code>User.findById</code>	Busca en BD si el usuario del token todavía existe
<code>done(null, user)</code>	Autenticación exitosa, Passport adjunta <code>user</code> a <code>req.user</code>
<code>done(null, false)</code>	Token válido pero usuario no existe en BD
<code>done(err, false)</code>	Error en la consulta

 Archivo: `models/user.js`

Ubicación: `backend/nodejs_express/models/user.js`

Función: Interactuar con la tabla `users` de MySQL

Código completo con explicación:

```
javascript

const db = require('../config/config');
const bcrypt = require('bcryptjs');
const User = {};
```

Línea	Explicación
<code>require('../config/config')</code>	Importa la conexión a MySQL
<code>require('bcryptjs')</code>	Para encriptar contraseñas
<code>User = {}</code>	Objeto vacío que contendrá todos los métodos

Método `findAll` - Obtener todos los usuarios

```
javascript

User.findAll = (result) => {
  const sql = `SELECT id, email, name, lastname, phone, image, role, created_at, u
pdated_at FROM users`;
  db.query(sql, (err, users) => {
    if (err) {
      console.log('Error al listar usuarios: ', err);
      result(err, null);
    } else {
      console.log('Usuarios encontrados: ', users.length);
      result(null, users);
    }
  })
}
```

```
});  
};
```

Explicación:

- Ejecuta `SELECT` (sin password por seguridad)
- `db.query()` ejecuta SQL y devuelve `users`
- Si hay error, llama `result(err, null)`
- Si éxito, llama `result(null, users)`

Método `findById` - Buscar por ID

javascript

```
User.findById = (id, result) => {  
  const sql = `SELECT id, email, name, lastname, image, phone, role, password FROM  
  users WHERE id = ?`;   
  db.query(sql, [id], (err, user) => {  
    if (err) {  
      console.log('Error al consultar: ', err);  
      result(err, null);  
    }  
    else {  
      console.log('Usuario consultado: ', user[0] );  
      result(null, user[0]);  
    }  
  });  
};
```

Explicación:

- `?` es un placeholder para prevenir SQL injection
- `[id]` reemplaza al `?`
- Devuelve `user[0]` (primer resultado) porque `id` es único

Método `findByEmail` - Buscar por email

javascript

```
User.findByEmail = (email, result) => {  
  const sql = `SELECT id, email, name, lastname, image, phone, role, password FROM  
  users WHERE email = ?`;   
  db.query(sql, [email], (err, user) => {  
    if (err) {  
      console.log('Error al consultar: ', err);  
      result(err, null);  
    }  
    else {  
      console.log('Usuario consultado: ', user[0] );  
      result(null, user[0]);  
    }  
  });  
};
```

Similar a `findById` pero busca por email (útil para login)

Método `create` - Registrar nuevo usuario

javascript

```

User.create = async (user, result) => {
  const hash = await bcrypt.hash(user.password, 10);
  const validRoles = ['admin', 'seller', 'customer', 'user'];
  const role = validRoles.includes(user.role) ? user.role : 'user';
  const sql = `INSERT INTO users(
    name, lastname, email, password, phone, image, role,
    created_at, updated_at
  ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)`;
  db.query(sql,
    [
      user.name,
      user.lastname,
      user.email,
      hash,
      user.phone,
      user.image,
      role,
      new Date(),
      new Date()
    ], (err, res) => {
    if (err) {
      console.log('Error al crear al Usuario: ', err);
      result(err, null);
    } else {
      console.log('Usuario creado: ', {id: res.insertId, ...user});
      result(null, {id: res.insertId, ...user});
    }
  })
};

```

Explicación paso a paso:

Paso	Explicación
<code>await bcrypt.hash(password, 10)</code>	Encripta la contraseña con 10 rondas de sal
<code>validRoles</code>	Lista de roles permitidos
<code>role = ...</code>	Si el rol no es válido, asigna 'user' por defecto
<code>INSERT INTO</code>	Guarda en BD (incluye fechas de creación/actualización)
<code>res.insertId</code>	MySQL devuelve el ID autoincremental generado
<code>{id: res.insertId, ...user}</code>	Combina el nuevo ID con los datos del usuario

Método `update` - Actualizar usuario

```

javascript

User.update = async (user, result) => {
  let fields = [];
  let values = [];

  if (user.password) {
    const hash = await bcrypt.hash(user.password, 10);
    fields.push("password = ?");
    values.push(hash);
  }

  if (user.email) {
    fields.push("email = ?");
    values.push(user.email);
  }
}

```

```

    }
    // ... (similar para name, lastname, phone, image, role)

    fields.push("updated_at = ?");
    values.push(new Date());

    const sql = `UPDATE users SET ${fields.join(", ")} WHERE id = ?`;
    values.push(user.id);

    db.query(sql, values, (err, res) => {
      if (err) {
        console.log('Error al actualizar usuario: ', err);
        result(err, null);
      } else {
        console.log('Usuario actualizado: ', { id: user.id, ...user });
        result(null, { id: user.id, ...user });
      }
    });
  });
};

```

Explicación:

- Construye dinámicamente la consulta `UPDATE` según qué campos vengan en `user`
- Si viene `password`, lo encripta antes
- Siempre actualiza `updated_at` con la fecha actual
- Ejemplo: si solo envía `name` y `email`, genera
`UPDATE users SET name = ?, email = ?, updated_at = ? WHERE id = ?`

Método `delete` - Eliminar usuario

```

javascript

User.delete = (id, result) => {
  const sql = `DELETE FROM users WHERE id = ?`;
  db.query(sql, [id], (err, res) => {
    if (err) {
      console.log('Error al eliminar usuario: ', err);
      result(err, null);
    } else {
      console.log('Usuario eliminado con id: ', id);
      result(null, res);
    }
  });
};

```

Explicación:

- Ejecuta `DELETE FROM users WHERE id = ?`
- `res` contiene información de filas afectadas (`res.affectedRows`)

Archivo: `middlewares/authMiddleware.js`

Ubicación: `backend/nodejs_express/middlewares/authMiddleware.js`

Función: Verificar token JWT y roles de usuario

Código:

```

javascript

```

```
const jwt = require("jsonwebtoken");
const keys = require("../config/keys");
```

Función `verifyToken`

```
javascript

function verifyToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  if (!authHeader) {
    return res.status(403).json({
      success: false,
      message: "No se proporcionó un token",
    });
  }

  const token = authHeader.split(" ")[1];
  if (!token) {
    return res.status(403).json({
      success: false,
      message: "Formato de token inválido",
    });
  }

  jwt.verify(token, keys.secretOrKey, (err, decoded) => {
    if (err) {
      return res.status(401).json({
        success: false,
        message: "Token inválido o expirado",
        error: err,
      });
    }
    req.user = decoded;
    next();
  });
}
```

Explicación paso a paso:

Paso	Explicación
<code>req.headers["authorization"]</code>	Obtiene el header <code>Authorization</code> (ej: "Bearer eyJhbG...")
Si no existe	Responde 403: "No se proporcionó un token"
<code>authHeader.split(" ")[1]</code>	Extrae solo el token (quita "Bearer ")
Si no hay token	Responde 403: "Formato inválido"
<code>jwt.verify(token, secret, callback)</code>	Verifica que el token sea válido y no haya expirado
Si error	Responde 401: "Token inválido o expirado"
Si éxito	<code>req.user = decoded</code> guarda los datos del token (id, email, role) en la request
<code>next()</code>	Continúa al siguiente middleware o al controlador

Función `authorizeRoles`

```
javascript
```



```
function authorizeRoles(roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Acceso denegado: se requiere rol ${roles.join(" o ")}`,
      });
    }
    next();
  };
}
```

Explicación:

- Es una **fábrica de middlewares**: recibe `roles` permitidos y devuelve un middleware
- Verifica que `req.user.role` esté dentro de `roles`
- Si no, responde 403 con mensaje de qué rol(es) se necesitan
- Si sí, llama `next()`

Ejemplo de uso:

```
javascript

authorizeRoles(['admin', 'seller']) // Solo admin o seller pueden pasar
```

Archivo: controllers/userController.js

Ubicación: backend/nodejs_express/controllers/userController.js

Función: Procesar peticiones HTTP y devolver respuestas JSON

Login

```
javascript

login(req, res) {
  const email = req.body.email;
  const password = req.body.password;

  User.findById(email, async (err, myUser) => {
    if (err) {
      return res.status(501).json({
        success: false,
        message: "Error al consultar el usuario",
        error: err,
      });
    }
  })

  if (!myUser) {
    return res.status(401).json({
      success: false,
      message: "El email no existe en la base de datos",
    });
  }

  const isPasswordValid = await bcrypt.compare(password, myUser.password);

  if (isPasswordValid) {
    const token = jwt.sign(
      {
        id: myUser.id,
        email: myUser.email,
        role: myUser.role,
```

```

        exp: Math.floor(Date.now() / 1000) + (60 * 60) // 1 hora
    },
    keys.secretOrKey
);

const data = {
    id: myUser.id,
    email: myUser.email,
    name: myUser.name,
    lastname: myUser.lastname,
    image: myUser.image,
    phone: myUser.phone,
    role: myUser.role,
    session_token: `JWT ${token}`,
};

return res.status(201).json({
    success: true,
    message: "Usuario autenticado",
    data: data,
});
} else {
    return res.status(401).json({
        success: false,
        message: "Contraseña o correo incorrecto",
    });
}
});
}

```

Explicación paso a paso:

Paso	Explicación
<code>req.body.email</code> y <code>req.body.password</code>	Obtiene credenciales del cuerpo de la petición
<code>User.findByEmail(email, callback)</code>	Busca usuario por email en BD
Si error BD	Responde 501 (error interno)
Si no existe usuario	Responde 401 (no autorizado)
<code>bcrypt.compare(password, hash)</code>	Compara contraseña plana con hash almacenado
Si contraseña incorrecta	Responde 401
Si contraseña correcta	Crea token JWT con <code>jwt.sign()</code>
<code>exp: ... + (60 * 60)</code>	Token expira en 1 hora (3600 segundos)
Devuelve JSON con <code>data</code> + <code>session_token</code>	El cliente usará este token para futuras peticiones

getAllUsers

```

javascript

getAllUsers(req, res) {
    User.findAll((err, users) => {
        if (err) {
            return res.status(501).json({
                success: false,
                message: "Error al listar usuarios",
                error: err,
            });
        }
    });
}

```

```

    });
  }
  return res.status(200).json({
    success: true,
    message: "Lista de usuarios",
    data: users,
  });
});
}

```

Explicación:

- Llama a `User.findAll()` del modelo
- Si éxito, responde 200 con array de usuarios
- Si error, responde 501

register

```

javascript

register(req, res) {
  const user = req.body;

  if (!user.role) {
    user.role = "user";
  }

  User.create(user, (err, data) => {
    if (err) {
      return res.status(501).json({
        success: false,
        message: "Error al crear al usuario",
        error: err,
      });
    } else {
      return res.status(201).json({
        success: true,
        message: "Usuario creado correctamente",
        data: data,
      });
    }
  });
}

```

Explicación:

- Toma `user` del cuerpo de la petición (`req.body`)
- Si no tiene `role`, asigna `"user"` por defecto
- Llama a `User.create()` que encripta la contraseña y guarda en BD
- Responde 201 (creado) con los datos del usuario (incluyendo ID generado)

getUserById, getUserUpdate, getUserDelete

Siguen el mismo patrón:

1. Extraer `id` de `req.params.id` (o `req.body` para update)
2. Llamar al método correspondiente del modelo
3. Responder con JSON según éxito/error

 Archivo: routes/userRoutes.js

Ubicación: backend/nodejs_express/routes/userRoutes.js

Función: Definir los endpoints HTTP y asignar controladores/middlewares

Código:

```
javascript

const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const { verifyToken, authorizeRoles } = require('../middlewares/authMiddleware');

// Rutas públicas
router.post('/create', userController.register);
router.post('/login', userController.login);

// Rutas protegidas
router.get('/', verifyToken, authorizeRoles(['admin', 'seller']), userController.getAllUsers);
router.get('/:id', verifyToken, authorizeRoles(['admin', 'seller']), userController.getUserById);
router.put('/:id', verifyToken, authorizeRoles(['admin', 'seller']), userController.getUserUpdate);
router.delete('/delete/:id', verifyToken, authorizeRoles(['admin']), userController.getUserDelete);

module.exports = router;
```

Explicación de cada ruta:

Método	Ruta	Middlewares	Controlador	¿Quién puede acceder?
POST	/create	ninguno	register	Cualquiera
POST	/login	ninguno	login	Cualquiera
GET	/	verifyToken + authorizeRoles(['admin', 'seller'])	getAllUsers	Admin o Seller
GET	/:id	verifyToken + authorizeRoles(['admin', 'seller'])	getUserById	Admin o Seller
PUT	/:id	verifyToken + authorizeRoles(['admin', 'seller'])	getUserUpdate	Admin o Seller
DELETE	/delete/:id	verifyToken + authorizeRoles(['admin'])	getUserDelete	Solo Admin

Orden de ejecución en ruta protegida:

```
text

1. router.get('/')
2. verifyToken (valida JWT y pone req.user)
3. authorizeRoles(['admin', 'seller']) (verifica rol)
4. userController.getAllUsers (ejecuta lógica)
5. Devuelve JSON
```

 Archivo: server.js

Ubicación: backend/nodejs_express/server.js

Función: Configurar la aplicación Express (middlewares globales, rutas, manejo de errores)

Código:

```
javascript

const express = require('express');
const logger = require('morgan');
const cors = require('cors');
const usersRoutes = require('./routes/userRoutes');

const app = express();

// Middlewares globales
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cors());

// Rutas
app.use('/api/users', usersRoutes);

// Endpoints de prueba
app.get('/', (req, res) => {
  res.send('Ruta raíz del Backend');
});

app.get('/test', (req, res) => {
  res.send('Ruta TEST');
});

// Manejo de errores
app.use((err, req, res, next) => {
  console.log(err);
  res.status(err.status || 500).send(err.stack);
});

module.exports = app;
```

Explicación línea por línea:

Línea	Explicación
<code>express()</code>	Crea la aplicación Express
<code>app.use(logger('dev'))</code>	Registra cada petición en consola (método, ruta, tiempo, código)
<code>app.use(express.json())</code>	Parse automático de JSON en <code>req.body</code>
<code>app.use(express.urlencoded({extended: true}))</code>	Parse datos de formularios URL-encoded
<code>app.use(cors())</code>	Permite peticiones desde otros dominios (evita bloqueos CORS)
<code>app.use('/api/users', usersRoutes)</code>	Monta las rutas de usuarios en el prefijo <code>/api/users</code>
<code>app.get('/') y /test</code>	Endpoints de prueba simples
<code>app.use((err, req, res, next))</code>	Middleware de errores: captura cualquier error no manejado

¿Por qué hay dos archivos (server.js e index.js)?

Archivo	Responsabilidad
server.js	Configuración de Express (middlewares, rutas)
index.js	Creación del servidor HTTP, puerto, host, CORS adicional

Esto separa la lógica de la aplicación de la configuración del servidor.

Archivo: index.js

Ubicación: backend/nodejs_express/index.js

Función: Crear y arrancar el servidor HTTP

Código:

```
javascript

const http = require('http');
const app = require('./server');
const cors = require('cors');
const port = process.env.PORT || 3000;
const host = process.env.HOST || 'localhost';

// Configuración CORS
app.use(cors({
  origin: [
    'http://10.1.196.11',
    'http://localhost:5173',
    'http://localhost:3000',
    'http://127.0.0.1:5173',
    'http://127.0.0.1:3000'
  ],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-Requested-With']
})));

// Manejar preflight CORS
app.options('*', cors());

app.set('port', port);

const server = http.createServer(app);

server.listen(port, host, () => {
  console.log(`Servidor corriendo en http://${host}:${port}`);
  console.log(`Accesible en la red local en http://${getLocalIp()}:${port}`);
});

function getLocalIp() {
  const { networkInterfaces } = require('os');
  const nets = networkInterfaces();
  for (const name of Object.keys(nets)) {
    for (const net of nets[name]) {
      if (net.family === 'IPv4' && !net.internal) {
        return net.address;
      }
    }
  }
  return 'localhost';
}
```

Explicación paso a paso:

Paso	Explicación
<code>http.createServer(app)</code>	Crea servidor HTTP usando la app de Express
<code>cors()</code> con <code>origin</code> explícito	Solo permite peticiones de esas URLs específicas (más seguro que <code>*</code>)
<code>credentials: true</code>	Permite enviar cookies/credenciales
<code>app.options('*', cors())</code>	Maneja peticiones OPTIONS (preflight) automáticamente
<code>server.listen(port, host, callback)</code>	Arranca el servidor en el puerto y host indicados
<code>getLocalIp()</code>	Detecta automáticamente la IP local de la máquina (ej: 192.168.1.x)

 Archivo: `package.json`

Ubicación: `backend/nodejs_express/package.json`

Función: Definir metadatos, dependencias y scripts del proyecto

```
json

{
  "name": "api_nodejs_express",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "dependencies": {
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "morgan": "^1.10.0",
    "mysql": "^2.18.1",
    "passport": "^0.7.0",
    "passport-jwt": "^4.0.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.11"
  }
}
```

Explicación de scripts:

Script	Comando	Qué hace
<code>start</code>	<code>node server.js</code>	Ejecuta la API en modo producción
<code>dev</code>	<code>nodemon index.js</code>	Ejecuta con nodemon (reinicio automático)
<code>test</code>	<code>echo ...</code>	Placeholder para pruebas unitarias

Archivo: `.env` (ejemplo)

Ubicación: `backend/nodejs_express/.env`

Función: Variables de entorno (no subir a Git)

```
env

PORT=3000
HOST=localhost

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña
DB_NAME=api_users
DB_PORT=3306

JWT_SECRET=clave_secreta_muy_segura
```

Mapa de dependencias entre archivos

```
text

index.js (servidor)
├── server.js
│   ├── routes/userRoutes.js
│   │   ├── controllers/userController.js
│   │   │   ├── models/user.js
│   │   │   │   └── config/config.js (MySQL)
│   │   │   └── config/keys.js (JWT)
│   │   └── middlewares/authMiddleware.js
│   │       ├── config/keys.js (JWT)
│   │       └── jsonwebtoken
│   └── config/passport.js
│       ├── config/keys.js
│       └── models/user.js
└── .env (variables globales)
```

¿Listo para el **siguiente** punto final: puesta en marcha de la API?

[siguiente](#)

Perfecto. Llegamos al último punto.

5) Puesta en marcha de la API

Aquí tienes una guía completa paso a paso para poner en funcionamiento la API desde cero hasta que esté totalmente operativa.

Lista de verificación previa (Pre-flight check)

Antes de empezar, asegúrate de tener:

Requisito	¿Lo tienes?	Cómo verificarlo
Node.js instalado	☐	<code>node --version</code>
npm instalado	☐	<code>npm --version</code>

Requisito	¿Lo tienes?	Cómo verificarlo
MySQL instalado y corriendo	☐	<code>mysql --version</code>
Base de datos <code>api_users</code> creada	☐	<code>mysql -u root -p -e "SHOW DATABASES;"</code>
Tabla <code>users</code> creada	☐	Verificar con <code>DESCRIBE users;</code>
Archivo <code>.env</code> configurado	☐	Revisar que exista en la raíz
Puerto 3000 libre	☐	<code>netstat -an findstr :3000</code> (Windows) o <code>lsof -i :3000</code> (Mac/Linux)

Paso 1: Preparación inicial

1.1 Abrir terminal en la carpeta del proyecto

```
bash

cd backend/nodejs_express
```

1.2 Verificar que estás en el lugar correcto

```
bash

ls -la
# Deberías ver: config/ controllers/ middlewares/ models/ routes/ index.js server.
js package.json .env
```

1.3 Instalar dependencias (si no lo hiciste antes)

```
bash

npm install
```

Tiempo estimado: 30-60 segundos

Possible error: Si ves `ERESOLVE unable to resolve dependency tree`, ejecuta:

```
bash

npm install --legacy-peer-deps
```

Paso 2: Configurar Base de Datos (si no está lista)

2.1 Conectar a MySQL

```
bash

mysql -u root -p
# Ingresa tu contraseña
```

2.2 Crear base de datos (si no existe)

```
sql

CREATE DATABASE IF NOT EXISTS api_users;
USE api_users;
```

2.3 Crear tabla users

```
sql

CREATE TABLE IF NOT EXISTS `users` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `lastname` VARCHAR(100) NOT NULL,
  `email` VARCHAR(150) NOT NULL UNIQUE,
  `password` VARCHAR(255) NOT NULL,
  `phone` VARCHAR(20) DEFAULT NULL,
  `image` VARCHAR(255) DEFAULT NULL,
  `role` ENUM('admin', 'seller', 'customer', 'user') DEFAULT 'user',
  `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
);
```

2.4 (Opcional) Insertar usuario admin de prueba

```
sql

-- Contraseña: admin123 (encriptada con bcrypt)
INSERT INTO users (name, lastname, email, password, phone, role) VALUES
('Admin', 'Sistema', 'admin@api.com', '$2a$10$YourEncryptedPasswordHere', '123456789', 'admin');

-- O inserta un usuario normal
INSERT INTO users (name, lastname, email, password, phone, role) VALUES
('Juan', 'Perez', 'juan@test.com', '$2a$10$N9qo8uLOickgx2ZMRZoMy.Mqrqk7qZxxXqKqKqKqKqKqKqKqKq', '123456789', 'user');
```

⚠ Para generar un hash de contraseña válido, puedes usar este código temporal en Node.js:

```
bash

node -e "console.log(require('bcryptjs').hashSync('admin123', 10))"
```

2.5 Salir de MySQL

```
sql

EXIT;
```

🔧 Paso 3: Configurar archivo .env

Crea o verifica el archivo `.env` en `backend/nodejs_express/.env`:

```
env

# Servidor
PORT=3000
HOST=localhost

# Base de datos
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_contraseña_real_aqui
DB_NAME=api_users
DB_PORT=3306

# JWT - ¡Cambia esto por una clave segura!
JWT_SECRET=mi_clave_super_segura_cambia_esto_por_algo_aleatorio
```

Recomendación: Para generar una clave JWT segura:

```
bash

node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

▶ Paso 4: Ejecutar la API

Opción A: Modo desarrollo (recomendado para pruebas)

```
bash

npm run dev
```

Opción B: Modo producción

```
bash

npm start
```

✅ **Éxito - Deberías ver:**

```
text

Base de datos conectada
Servidor corriendo en http://localhost:3000
Accesible en la red local en http://192.168.1.X:3000
```

❌ **Error común - Puerto ocupado:**

```
text

Error: listen EADDRINUSE: address already in use :::3000
```

Solución: Cambia el puerto en `.env` o mata el proceso:

```
bash

# Encontrar proceso en puerto 3000
lsof -i :3000 # Mac/Linux
netstat -ano | findstr :3000 # Windows

# Matar proceso (Linux/Mac)
kill -9 [PID]
```

🔧 Paso 5: Probar la API (desde otra terminal)

5.1 Probar endpoints públicos

Endpoint raíz:

```
bash

curl http://localhost:3000/
```

✅ **Respuesta esperada:** Ruta raíz del Backend

Endpoint test:

```
bash
```

```
curl http://localhost:3000/test
```

Respuesta esperada: Ruta TEST

Registrar usuario:

```
bash

curl -X POST http://localhost:3000/api/users/create \
-H "Content-Type: application/json" \
-d '{
  "name": "Maria",
  "lastname": "Gomez",
  "email": "maria@test.com",
  "password": "123456",
  "phone": "987654321",
  "role": "customer"
}'
```

✓ Respuesta esperada:

```
json

{
  "success": true,
  "message": "Usuario creado correctamente",
  "data": {
    "id": 1,
    "name": "Maria",
    "lastname": "Gomez",
    "email": "maria@test.com",
    "password": "123456",
    "phone": "987654321",
    "role": "customer"
  }
}
```

Login:

```
bash

curl -X POST http://localhost:3000/api/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "maria@test.com",
  "password": "123456"
}'
```

✓ Respuesta esperada:

```
json

{
  "success": true,
  "message": "Usuario autenticado",
  "data": {
    "id": 1,
    "email": "maria@test.com",
    "name": "Maria",
    "lastname": "Gomez",
    "session_token": "JWT_eyJhbGciOiJIUzI1NiIs..."
  }
}
```

🔑 Guarda el token que recibes, lo necesitarás para las rutas protegidas.

5.2 Probar rutas protegidas (requieren token)

Obtener todos los usuarios (solo admin/seller):

```
bash

curl http://localhost:3000/api/users/ \
-H "Authorization: Bearer TU_TOKEN_AQUI"
```

Obtener usuario por ID:

```
bash

curl http://localhost:3000/api/users/1 \
-H "Authorization: Bearer TU_TOKEN_AQUI"
```

Actualizar usuario:

```
bash

curl -X PUT http://localhost:3000/api/users/1 \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TU_TOKEN_AQUI" \
-d '{
  "name": "Maria Actualizada",
  "phone": "111222333"
}'
```

Eliminar usuario (solo admin):

```
bash

curl -X DELETE http://localhost:3000/api/users/delete/1 \
-H "Authorization: Bearer TU_TOKEN_ADMIN"
```

 Paso 6: Solución de problemas comunes

Problema	Posible causa	Solución
Cannot find module 'express'	Dependencias no instaladas	<code>npm install</code>
ER_ACCESS_DENIED_ERROR	Contraseña MySQL incorrecta	Verifica <code>DB_PASSWORD</code> en <code>.env</code>
ER_BAD_DB_ERROR	Base de datos no existe	Crea <code>api_users</code> con <code>CREATE DATABASE</code>
ER_NO_SUCH_TABLE	Tabla no creada	Ejecuta el <code>CREATE TABLE</code> del paso 2.3
JWT_SECRET is required	Falta clave en <code>.env</code>	Agrega <code>JWT_SECRET=...</code>
401 Unauthorized	Token inválido o expirado	Vuelve a hacer login y usa el nuevo token
403 Forbidden	Rol insuficiente	Usa un usuario con rol <code>admin</code> o <code>seller</code>
Token expired	Pasó más de 1 hora	Vuelve a hacer login

Paso 7: Probar con Postman (alternativa visual)

Si prefieres una interfaz gráfica:

1. **Abre Postman**
2. **Crear nueva colección llamada "API Usuarios"**
3. **Configurar variables de entorno en Postman:**
 - `base_url` : `http://localhost:3000`
 - `token` : (vacío, se llenará después del login)
4. **Endpoints para crear:**

Método	URL	Headers	Body (JSON)
POST	<code>{{base_url}}/api/users/create</code>	Content-Type: application/json	<code>{"name":"Juan","lastname":"Pere est.com","password":"123456"}</code>
POST	<code>{{base_url}}/api/users/login</code>	Content-Type: application/json	<code>{"email":"juan@test.com","passw</code>
GET	<code>{{base_url}}/api/users</code>	Authorization: Bearer <code>{{token}}</code>	-
GET	<code>{{base_url}}/api/users/1</code>	Authorization: Bearer <code>{{token}}</code>	-

5. **Crear test de login para guardar token automáticamente:**

```
javascript

// En la pestaña "Tests" del Login
if (pm.response.code === 201) {
  var jsonData = pm.response.json();
  pm.environment.set("token", jsonData.data.session_token.replace("JWT ", ""));
}
```

Paso 8: Puesta en marcha para producción

Si quieres desplegar la API en un servidor real:

8.1 Configurar variables de entorno de producción

```
env

PORT=8080
HOST=0.0.0.0
DB_HOST=produccion-mysql-server.com
DB_USER=usuario_produccion
DB_PASSWORD=contraseña_fuerte
DB_NAME=api_users_prod
JWT_SECRET=clave_muy_segura_unica_para_produccion
```

8.2 Usar PM2 (Process Manager para Node.js)

```
bash

# Instalar PM2
npm install -g pm2

# Iniciar API con PM2
pm2 start index.js --name "api-usuarios"

# Ver estado
pm2 status

# Ver Logs
pm2 logs api-usuarios
```

```
# Configurar inicio automático
pm2 startup
pm2 save
```

8.3 Configurar Nginx como proxy inverso (opcional)

```
nginx

server {
    listen 80;
    server_name api.tudominio.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

✅ Paso 9: Verificación final - Lista de chequeo

Marcas todo lo siguiente:

Verificación	Estado
☐ Servidor corre sin errores	☐
☐ Endpoint <code>GET /</code> responde "Ruta raíz"	☐
☐ Endpoint <code>POST /api/users/create</code> crea usuarios	☐
☐ Endpoint <code>POST /api/users/login</code> devuelve token	☐
☐ Endpoint <code>GET /api/users</code> funciona con token válido	☐
☐ Endpoint <code>GET /api/users/:id</code> funciona con token	☐
☐ Endpoint <code>PUT /api/users/:id</code> actualiza datos	☐
☐ Endpoint <code>DELETE /api/users/delete/:id</code> (solo admin)	☐
☐ Roles funcionan (user no puede listar, admin sí)	☐
☐ Token expira después de 1 hora	☐

🎉 ¡Felicidades!

Tu API está completamente operativa. Ahora puedes:

- Conectarla a un frontend (React, Vue, Angular, etc.)
- Integrarla con aplicaciones móviles
- Documentarla con Swagger (completando `config/swagger.js`)
- Agregar más endpoints (productos, pedidos, etc.)
- Implementar pruebas unitarias con Jest

Resumen rápido de comandos útiles

```
bash

# Instalación y ejecución
npm install           # Instalar dependencias
npm run dev           # Modo desarrollo (con nodemon)
npm start             # Modo producción

# Pruebas rápidas con curl
curl http://localhost:3000/test           # Test básico
curl -X POST http://localhost:3000/api/users/login -H "Content-Type: application/json" -d '{"email":"admin@test.com","password":"123456"}'

# Gestión con PM2 (producción)
pm2 start index.js --name api
pm2 logs api
pm2 restart api
pm2 stop api
pm2 delete api

# Solución de problemas
lsof -i :3000           # Ver qué usa el puerto 3000 (Mac/Linux)
netstat -ano | findstr :3000 # Windows
```

¿Necesitas ayuda con algún paso en particular o quieres que explique cómo conectar esta API a un frontend (React, Vue, etc.)?